SIMATS ENGINEERING ASSESSMENT
TOOL 2 : Scenario-Based

COURSE CODE: CSA1516

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Total Marks: 25

Dave's Taxonomy: Manipulation (Level 2) Question
Count: 5

Code of Conduct

I JAFFRIN N (192521200) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Assessment Tasks
Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

Scenario

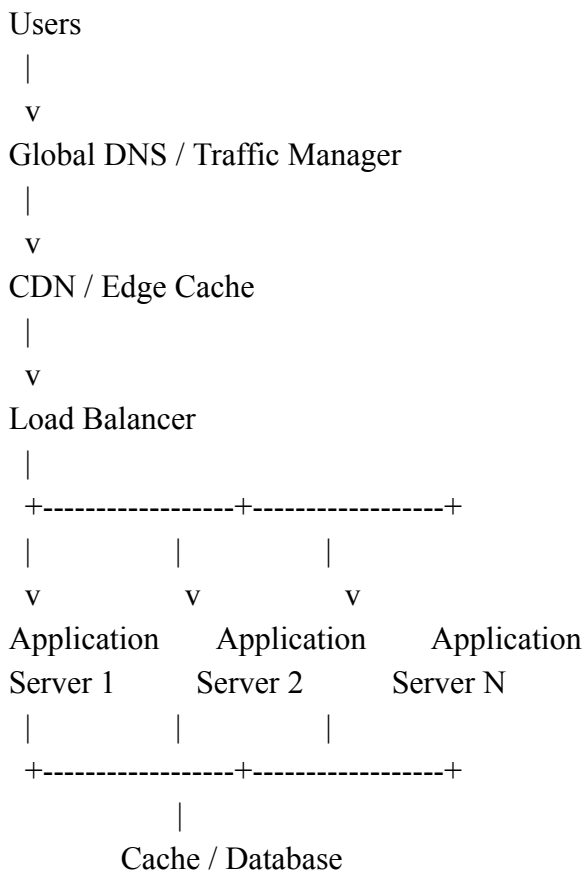
A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering, low latency, high availability, and sudden traffic spikes.

Proposed Cloud Architecture

The media company should use a cloud-based architecture consisting of:

- Global DNS or Traffic Manager
- Content Delivery Network (CDN)
- Edge caching
- Load balancer
- Multiple application servers
- Auto-scaling
- Video transcoding service
- Cloud object storage
- Application-level cache
- Scalable database with replication
- Multi-region deployment

Architecture Flow



Video Upload
|
v
Video Transcoding Service
|
v
Cloud Object Storage
|
v
CDN / Edge Servers
|
v
Users

1. CDN and Edge Caching

A Content Delivery Network (CDN) stores copies of popular video content at edge locations close to users.

Instead of every user accessing the central cloud storage:

User → Central Server → Video

the user receives the video from the nearest CDN server:

User → Nearest CDN Edge Server → Video

Benefits

- Reduces latency
- Minimizes buffering
- Reduces load on the main server
- Improves global performance
- Supports millions of concurrent users

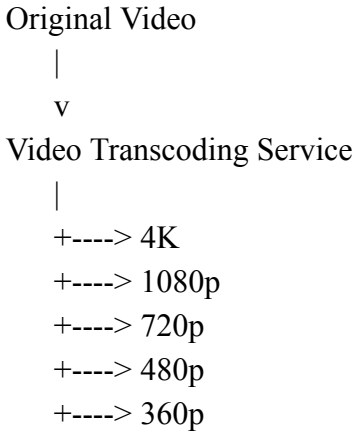
For example, if one million users watch the same movie, the CDN can deliver cached copies instead of forcing the main server to process one million separate requests.

1.2. Video Transcoding

Videos are converted into multiple resolutions such as:

- 4K

- 1080p
- 720p
- 480p
- 360p



This allows the streaming platform to support users with different devices and internet speeds.

1.3. Adaptive Bitrate Streaming

The player automatically switches to an appropriate video quality based on the user's network speed.

For example:

High Internet Speed → 1080p

Network Speed Drops → 720p

Very Slow Network → 480p

Instead of stopping the video and causing buffering, the system reduces the video quality.

Benefits

- Continuous playback
- Reduced buffering
- Better user experience
- Efficient use of network bandwidth

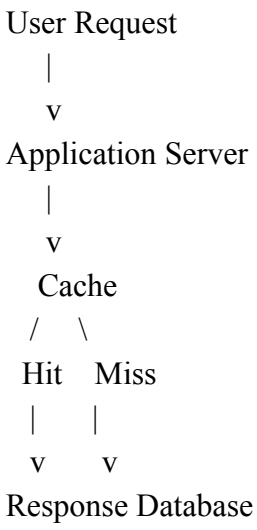
1.4. Application-Level Caching

Frequently accessed information such as:

- User sessions

- Trending videos
- Movie details
- Recommendations
- Search results
- Thumbnails

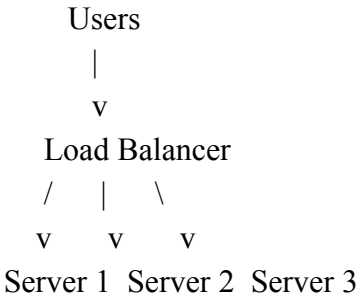
is stored in a fast cache.



A **cache hit** means the requested data is already available in the cache. This reduces repeated database access and improves response speed.

Load Balancing

A load balancer distributes incoming requests across multiple application servers.



If one server becomes overloaded, new requests are directed to less-loaded servers. Health checks also prevent traffic from being sent to failed servers.

Load Balancing Methods

- **Round Robin:** Requests are distributed equally among servers.
- **Least Connections:** New requests are sent to the server with the fewest active connections.

- **Geographic Load Balancing:** Users are directed to the nearest cloud region.
- **Weighted Load Balancing:** More powerful servers handle more traffic.

This prevents any single server from becoming a bottleneck.

Auto-Scaling

Auto-scaling automatically increases or decreases the number of application servers according to demand.

For example:

Normal Traffic
5 Servers

High Traffic
5 → 20 → 50 → 100 Servers

When traffic decreases:

100 → 50 → 20 → 5 Servers

Example Scaling Rules

If CPU usage remains above **70% for five minutes**, new application instances can be created.

If CPU usage remains below **30% for fifteen minutes**, unnecessary instances can be removed.

Auto-scaling can monitor:

- CPU utilization
- Memory usage
- Number of active users
- Requests per second
- Network traffic
- Response time
- Queue length

How Load Balancing and Auto-Scaling Work Together

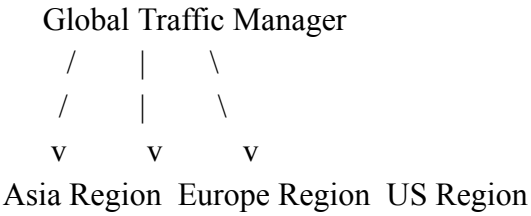
1. Traffic suddenly increases.
2. The load balancer distributes existing traffic across available servers.
3. Monitoring detects high resource usage.
4. Auto-scaling creates additional servers.
5. New servers pass health checks.
6. The load balancer begins sending traffic to the new servers.

7. When demand decreases, extra servers are removed.

This combination ensures that the streaming platform can handle sudden increases in users without causing major delays or buffering.

High Availability and Multi-Region Deployment

The platform should be deployed in multiple cloud regions.



Each region can contain:

- Application servers
- Load balancers
- Cache
- Database replicas

If one region fails, traffic can automatically be redirected to another healthy region.

Benefits

- High availability
- Disaster recovery
- Lower latency
- Better fault tolerance
- Continuous service during regional failures

Cost Optimization Strategies

A. Use CDN Caching

A CDN reduces repeated requests to the origin server and decreases bandwidth costs.

B. Use Auto-Scaling

The company only uses additional computing resources when traffic is high.

C. Use Reserved Capacity

For predictable workloads, long-term reserved or committed cloud resources can reduce costs.

D. Use Storage Tiering

Frequently Watched Videos → High-Performance Storage

Less Frequently Accessed Videos → Standard Storage

Old Videos → Low-Cost Archive Storage

1.5. Optimize Video Encoding

Efficient video compression reduces:

- Storage requirements
- Network bandwidth
- CDN costs

1.6. Use Serverless Services

Serverless or managed services can be used for temporary tasks such as:

- Thumbnail generation
- Notifications
- Metadata processing
- Analytics

1.7. Remove Idle Resources

The company should continuously monitor and remove:

- Unused virtual machines
- Unused storage
- Old snapshots
- Idle databases
- Over-provisioned resources

Conclusion

A combination of CDN, caching, video transcoding, adaptive bitrate streaming, load balancing, auto-scaling, multi-region deployment, scalable cloud storage, and database replication can support millions of concurrent users with minimal buffering and low latency.

Load balancing distributes user requests efficiently, while auto-scaling automatically adds servers during sudden traffic spikes and removes them when demand decreases. Caching and CDNs reduce repeated access to the origin server and improve streaming performance.

Therefore, this cloud architecture provides a scalable, highly available, fault-tolerant, and cost-efficient

solution for a high-demand video streaming platform.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

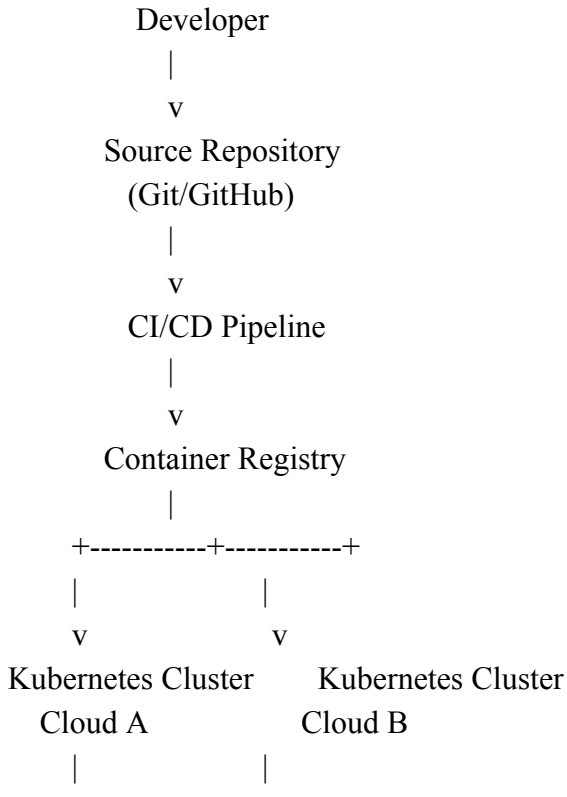
Scenario

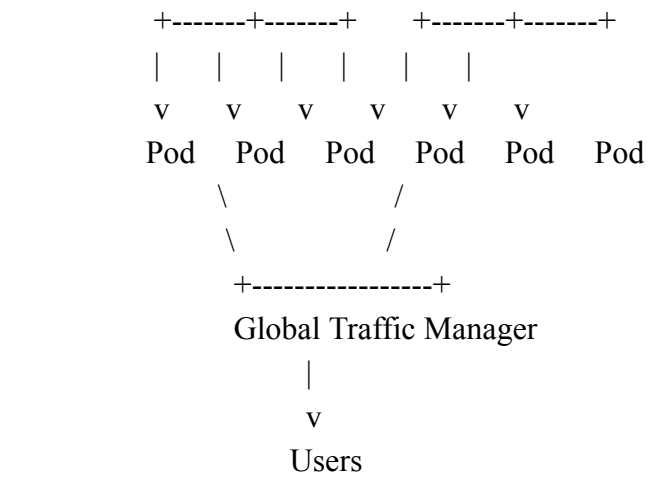
A fintech startup is deploying its applications using containers across multiple cloud providers. The startup faces challenges in:

- Maintaining consistency between cloud environments
- Managing application deployments
- Discovering and connecting microservices
- Scaling applications during high demand
- Handling failures
- Ensuring high availability and security

A suitable solution is to use container orchestration with Kubernetes, along with centralized CI/CD, Infrastructure as Code (IaC), service discovery, auto-scaling, and global traffic management.

A. Proposed Multi-Cloud Architecture





Architecture Flow

Developer
|
v
Source Repository
|
v
CI/CD Pipeline
|
v
Container Registry
|
+-------------------------+
| |
v v
Kubernetes Cluster Kubernetes Cluster
Cloud A Cloud B
| |
+------------+-------------+
|
v
Global Traffic Manager
|
v
Users

B. Container Orchestration Using Kubernetes

A container is a lightweight package that contains an application and everything required to run it.

For example:

Payment Application
+
Required Libraries
+
Configuration
=
Container

When many containers are used, managing them manually becomes difficult.

Therefore, Kubernetes is used as a **container orchestration platform**.

Kubernetes manages:

- Container deployment
- Container scheduling
- Automatic restart
- Health monitoring
- Service discovery
- Load balancing
- Auto-scaling
- Rolling updates
- Rollbacks

C. How Kubernetes Handles Container Failure

Suppose the application is running using three pods:

Kubernetes Cluster
|
+---- Pod 1 → Running
|
+---- Pod 2 → Running
|
+---- Pod 3 → Running

If Pod 2 fails:

Pod 2 Failed
|

v

Kubernetes Health Check Detects Failure

|

v

Kubernetes Removes Failed Pod

|

v

New Pod Created Automatically

Therefore:

Failed Container

↓

Failure Detected

↓

Automatic Recovery

↓

New Container/Pod Created

This improves application availability.

D. Maintaining Consistency Across Multiple Clouds

The same application should run consistently in Cloud A and Cloud B.

The startup should use:

- Standardized container images
- Infrastructure as Code
- Kubernetes deployment files
- Centralized source control
- Automated CI/CD pipelines
- Version-controlled configuration

Example

Developer

|

v

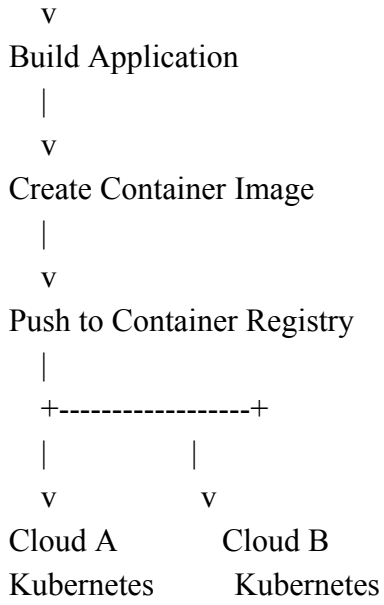
Git Repository

|

v

CI/CD Pipeline

|



The same application image and deployment configuration can be deployed across multiple Kubernetes clusters.

This ensures:

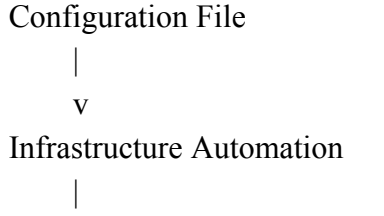
- Same application version
- Same configuration
- Reduced manual errors
- Faster deployment
- Easier rollback

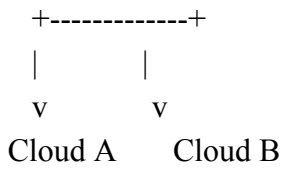
E. Infrastructure as Code

Infrastructure as Code means infrastructure is defined using configuration files rather than manually creating resources.

For example, the configuration can define:

- Virtual machines
- Networks
- Kubernetes clusters
- Storage
- Load balancers
- Security rules





Benefits

- Consistent infrastructure
- Repeatable deployment
- Faster recovery
- Reduced human errors
- Easy auditing

F. Service Discovery

In a microservices architecture, multiple services communicate with each other.

For example:

User Service
Payment Service
Transaction Service
Notification Service

If services use fixed IP addresses, problems occur when containers are restarted.

Example:

Old Pod IP = 10.10.10.5

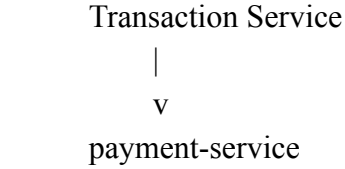
Pod fails and is recreated

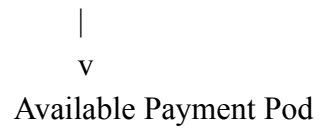
New Pod IP = 10.10.10.8

Other services would fail if they still try to communicate with the old IP address.

Instead, Kubernetes provides **service discovery**.

Example

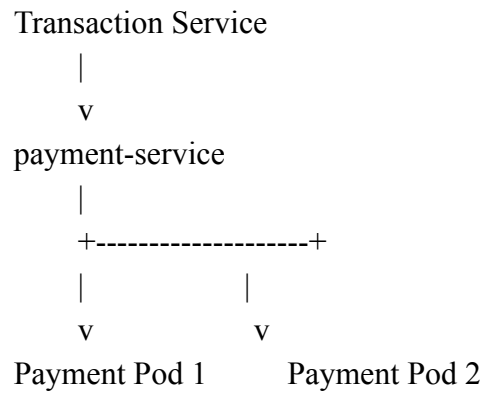




The service discovery mechanism identifies healthy service instances automatically.

If a pod is replaced and receives a new IP address, the service name remains unchanged.

Therefore:



The Kubernetes service routes traffic to a healthy available pod.

Benefits

- No hard-coded IP addresses
- Automatic discovery of services
- Improved fault tolerance
- Easy scaling
- Better microservice communication

G. Scaling

The application should automatically scale according to workload.

Scaling can be based on:

- CPU usage
- Memory usage
- Request rate
- Queue length
- Custom business metrics

Horizontal Scaling

Normal Traffic

|
v
3 Pods

During high demand:

3 Pods
↓
10 Pods
↓
30 Pods

When traffic decreases:

30 Pods
↓
10 Pods
↓
3 Pods

Two Types of Scaling

1. Pod Scaling

Adds more containers or pods.

High CPU Usage
|
v
Auto-Scaler
|
v
Create More Pods

2. Cluster Scaling

If the existing servers cannot support more pods:

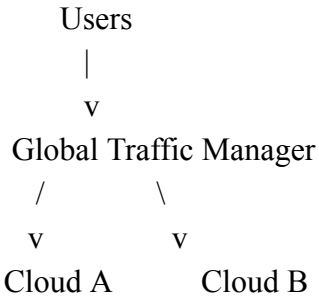
No Space for New Pods
|
v

Add More Worker Nodes
|
v
Create Additional Pods

This provides efficient use of resources.

H. Global Traffic Management

A global traffic manager directs users to the most appropriate cloud or region.



Traffic decisions can be based on:

- Cloud availability
- Geographic location
- Server health
- Latency
- Current load

If Cloud A fails:

Cloud A Failure
|
v
Health Check Detects Failure
|
v
Traffic Redirected
|
v
Cloud B

I. Benefits of Multi-Cloud Deployment

1. Avoids Vendor Lock-In

The company is not fully dependent on one cloud provider.

2. High Availability

If one provider has a major outage, another cloud can continue running the application.

3. Better Geographic Coverage

Different cloud providers may have infrastructure in different locations.

4. Cost Flexibility

The company can choose services and resources based on:

- Price
- Performance
- Availability

5. Disaster Recovery

A second cloud provider can act as an additional disaster recovery environment.

J. Risks and Challenges of Multi-Cloud

1. Increased Complexity

Managing multiple cloud platforms is more difficult.

2. Data Transfer Costs

Moving data between cloud providers can create significant costs.

3. Security Differences

Different providers may use different:

- Identity systems
- Security configurations
- Network models

4. Network Latency

Communication between services located in different clouds can be slower.

5. Operational Skills

The organization requires employees who understand:

- Kubernetes

- Cloud networking
- Security
- Multiple cloud platforms
- CI/CD tools

Conclusion

The fintech startup should use Kubernetes-based container orchestration, standardized container images, Infrastructure as Code, centralized CI/CD, service discovery, autoscaling, and global traffic management.

The same application image and deployment configuration can be deployed consistently across multiple Kubernetes clusters. Kubernetes automatically detects failed pods and creates replacements, while service discovery allows services to communicate using stable names instead of changing IP addresses.

Multi-cloud deployment improves resilience, availability, geographic coverage, and vendor flexibility. However, it also introduces complexity, security challenges, network latency, and data transfer costs. Therefore, strong governance, monitoring, automation, and operational discipline are essential.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.
   Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
   Provide steps to ensure business continuity during failures.

Scenario

An e-learning platform experiences data loss due to accidental deletion in the cloud.

The platform must protect:

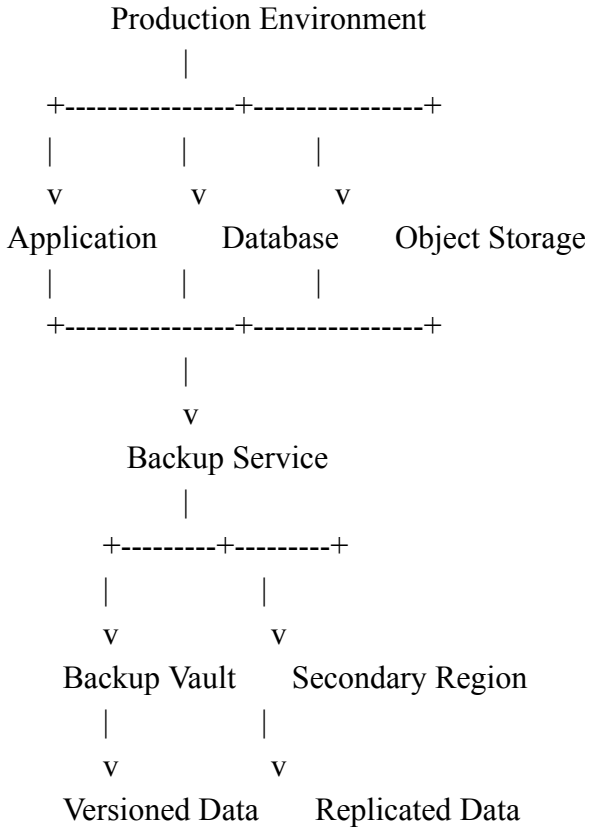
- Student records
- Course materials
- Assignments
- Video content
- Databases
- Application configurations

The organization needs a robust solution using:

- Automated backups
- Replication
- Versioning
- Disaster recovery
- Failover

- Monitoring

A. Proposed Backup and Disaster Recovery Architecture



B. Cloud-Native Backup Strategy

The organization should use automated backups for all important resources.

Types of Backups

- Full backups
- Incremental backups
- Database backups
- Database snapshots
- Transaction log backups
- Application configuration backups
- Infrastructure configuration backups

A typical backup policy can include:

Daily → Incremental Backup

Weekly → Full Backup

Monthly → Long-Term Backup

Important Requirement

Backups should be stored separately from the production environment and protected from:

- Accidental deletion
- Unauthorized modification
- Ransomware
- Data corruption

This ensures that if the production system is damaged, a clean backup is available.

C. Immutable Backup Storage

Critical backups should be protected from modification or deletion for a defined retention period.

Production Data
|
v
Backup Created
|
v
Immutable Backup Storage
|
v
Cannot Be Modified Easily

This provides additional protection against:

- Accidental deletion
- Malicious activity
- Ransomware
- Unauthorized userS

D. Object Storage Versioning

Versioning allows multiple versions of the same file to be stored.

Example

Version 1 → Original course file

Version 2 → Updated course file

Delete Marker → Latest state

The administrator can restore an earlier version when a file is accidentally deleted.

Example Scenario

CourseNotes.pdf
|
v
Accidentally Deleted
|
v
Administrator Checks Version History
|
v
Restore Version 1

This is useful for:

- Course documents
- Student assignments
- Video files
- Study materials

E. Cross-Region Replication

Critical data should be replicated to another cloud region.

Primary Region
|
| Replication
v
Secondary Region

The secondary region maintains a copy of critical:

- Databases
- Files
- Object storage
- Application data

This protects the organization against:

- Data center failure
- Regional outages

- Major disasters

F. RTO and RPO

Recovery Time Objective (RTO)

RTO is the maximum acceptable time required to restore services after a failure.

> **Example**
> RTO = 1 hour

This means the application must be restored and operational within **one hour**.

To achieve a low RTO, the organization can use:

- Warm standby infrastructure
- Preconfigured secondary systems
- Automated failover
- Infrastructure as Code
- Disaster recovery runbooks
- Automated recovery procedures

Recovery Point Objective (RPO)

RPO is the maximum acceptable amount of data loss measured in time.

> **Example**
> RPO = 5 minutes

This means no more than **five minutes of data** should be lost.

To achieve a low RPO, the organization can use:

- Continuous database replication
- Frequent backups
- Transaction log backups
- Frequent snapshots
- Cross-region replication

Database Update
|
v
Primary Database
|
v
Continuous Replication

|
v
Secondary Database

G. Steps for Business Continuity During Failure

Step 1: Detect the Failure

Monitoring detects:

- Application failure
- Database failure
- Storage failure
- Regional outage
- Data corruption

Step 2: Isolate the Problem

Prevent further data loss or corruption.

Step 3: Select the Recovery Point

Choose:

- Latest safe backup
- Previous object version
- Database snapshot
- Secondary replica

Step 4: Restore Data

Backup
|
v
Restore Data
|
v
Verify Integrity

Step 5: Fail Over if Required

Primary Region Failure
|
v
Health Check
|

v

Traffic Redirected

|

v

Secondary Region

Step 6: Verify Services

Check:

- Student records
- Course materials
- Database consistency
- Application functionality

Step 7: Continue Operations

Students and teachers should continue using the recovered system.

Step 8: Restore the Primary Environment

After the failure is resolved:

- Repair the primary environment
- Synchronize updated data
- Restore normal traffic
- Review the incident

Conclusion

A robust disaster recovery solution combines automated backups, immutable backup storage, object versioning, cross-region replication, database snapshots, continuous replication, and automated failover.

RTO is achieved by maintaining recovery infrastructure and automating restoration and failover. RPO is achieved through frequent backups and continuous replication.

These mechanisms protect data and ensure business continuity even during accidental deletion, infrastructure failure, or regional outages.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.

Discuss challenges in managing hybrid cloud environments.

Scenario

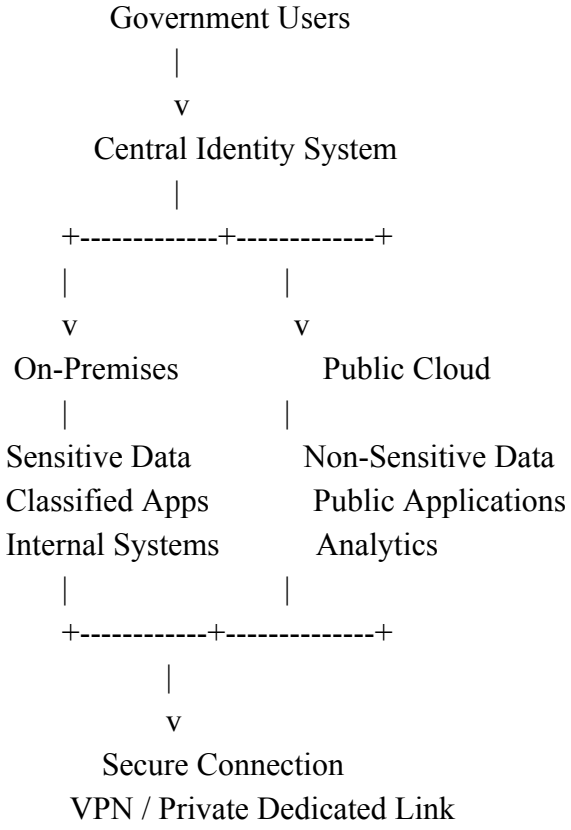
A government agency is adopting a hybrid cloud model.

Some applications and data must remain **on-premises** because of:

- Compliance requirements
- Data sovereignty
- Government regulations
- Security requirements

Other workloads can be moved to the public cloud.

A. Proposed Hybrid Cloud Architecture



B. Data Classification

Before migration, the agency should classify its data.

Sensitive Data – Keep On-Premises

Examples:

- Citizen records
- Confidential documents

- Classified information
- Sensitive databases
- Internal government systems

Non-Sensitive Data – Can Use Public Cloud

Examples:

- Public websites
- Information portals
- Public applications
- Non-sensitive analytics
- Development environments
- Testing environments

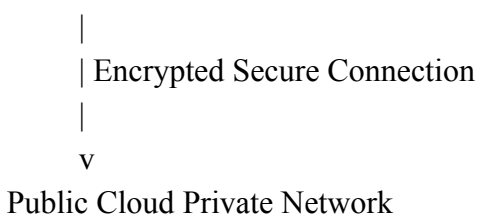
This approach ensures that sensitive data remains within the required compliance boundary.

C. Secure Communication Between Environments

The on-premises environment and cloud environment should communicate using secure private connections.

Possible options include:

- VPN tunnels
- Dedicated private connectivity
- Private network links
- Private endpoints

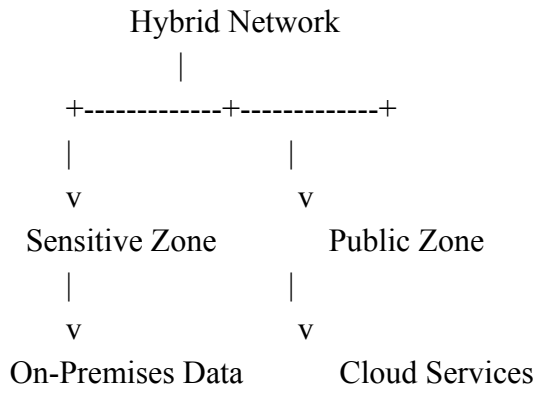


Additional security should include:

- Firewalls
- Network segmentation
- Security groups
- Network access control
- Intrusion detection and logging

D. Network Segmentation

The network should be divided into separate zones.

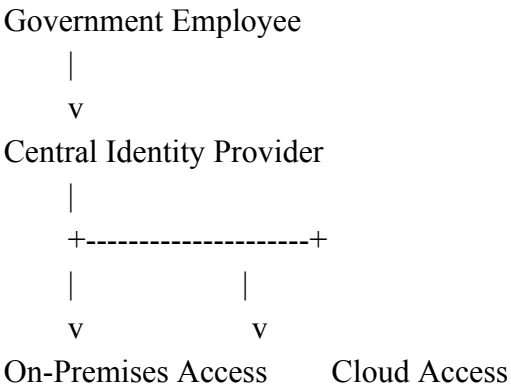


Sensitive systems should not be directly exposed to public networks.

Access should be allowed only through approved and monitored connections.

E. Identity Management

A centralized identity management system should control access to both cloud and on-premises resources.



Recommended controls:

- Single Sign-On (SSO)
- Multi-Factor Authentication (MFA)
- Role-Based Access Control (RBAC)
- Least privilege access
- Centralized auditing
- Periodic access reviews

F. Role-Based Access Control Example

v
Authorized Administrative Access

Analyst
|
v
Analytics-Related Access Only

Employee
|
v
Limited Application Access

This prevents users from accessing information that is not required for their job.

G. Encryption

Encryption in Transit

Data moving between:

- Users and applications
- On-premises and cloud
- Services and databases

should be protected using secure encrypted communication.

On-Premises
|
| Encrypted Data
v
Public Cloud

Encryption at Rest

Databases, object storage, backups, and virtual disks should be encrypted.

This protects data even if the physical storage system is compromised.

H. Key Management

Encryption keys should be:

- Centrally controlled
- Securely stored

- Rotated regularly
- Access restricted
- Audited

Encrypted Data
|
v
Central Key Management System
|
v
Authorized Access Only

Good key management is essential because encrypted data is only as secure as the keys used to protect it.

I. Challenges in Managing Hybrid Cloud Environments

1. Increased Complexity

Both cloud and on-premises environments must be managed together.

2. Network Latency

Communication between environments may introduce delays.

3. Data Synchronization

Keeping data consistent across environments can be difficult.

4. Security Consistency

Security policies must be applied consistently across both environments.

5. Monitoring Complexity

The organization requires centralized monitoring for:

- Cloud systems
- On-premises systems
- Networks
- Security events

6. Compliance Management

The agency must continuously ensure that data and applications meet regulatory requirements.

7. Configuration Management

Configuration differences between cloud and on-premises systems can create operational problems.

Conclusion

A secure hybrid cloud should keep sensitive workloads in the required on-premises environment while moving appropriate workloads to the public cloud.

The hybrid architecture should use secure private connectivity, centralized identity management, MFA, RBAC, least-privilege access, encryption in transit and at rest, secure key management, network segmentation, logging, and unified monitoring.

This approach allows the government agency to gain the scalability and flexibility of cloud computing while continuing to meet strict security and compliance requirements.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario

A SaaS (Software as a Service) provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if an entire cloud region goes down. The application may serve thousands or millions of users, so downtime, data loss, and service interruption must be minimized.

To achieve this, the SaaS provider should use a **multi-region cloud architecture** with:

- Multiple cloud regions
- Global traffic management
- Load balancing
- Application replication
- Database replication
- Automatic failover
- Continuous monitoring
- Real-time alerting
- Backup and disaster recovery mechanisms

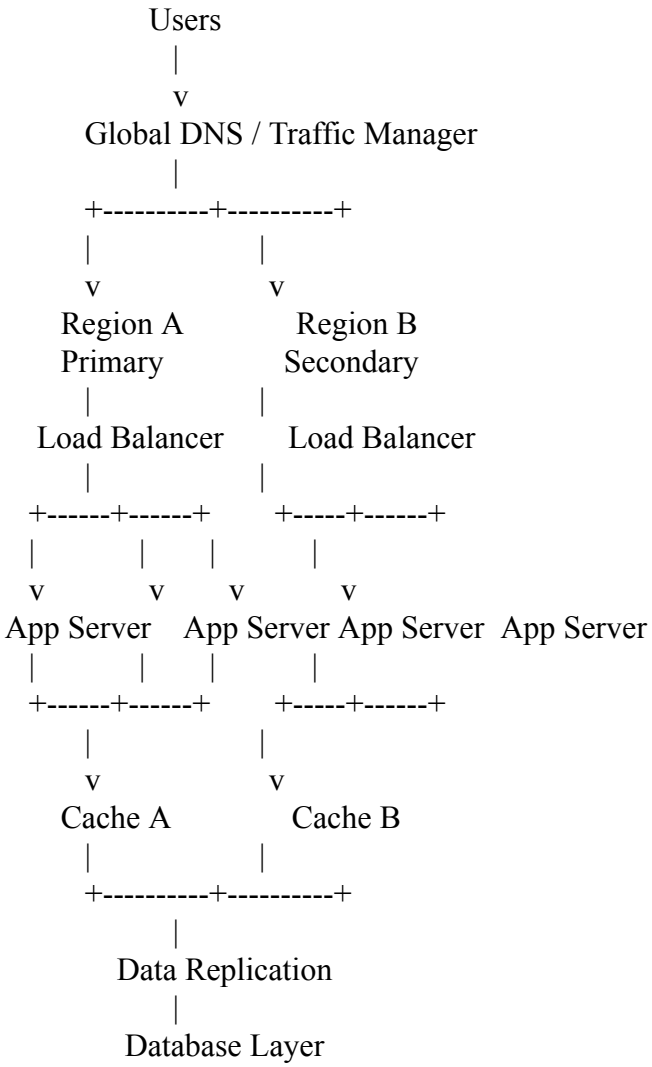
The main objective is to ensure that if one region fails, users are automatically redirected to another healthy region.

A. Proposed Multi-Region Deployment Architecture

A multi-region deployment means that the same application is deployed in two or more geographically

separate cloud regions.

Architecture Flow



The users first connect to a **Global DNS or Global Traffic Manager**. The traffic manager checks which region is healthy and routes users to the most appropriate region.

B. Multi-Region Deployment Strategies

There are two main strategies:

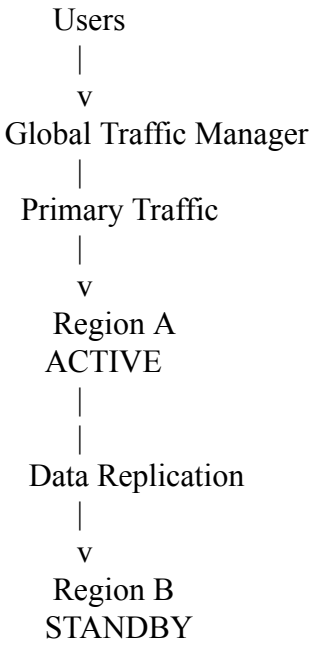
1. Active-Passive Deployment

In an active-passive model:

- Region A actively serves users.
- Region B remains available as a backup.

- If Region A fails, traffic is redirected to Region B.

Architecture



During Normal Operation

Users → Region A
Region A → Database A
Database A → Replicates Data → Region B

During Failure

Region A Fails
|
v
Failure Detected
|
v
Global Traffic Manager
|
v
Redirect Traffic to Region B
|
v
Region B Becomes Active

Advantages

- Easier to manage

- Lower complexity
- Easier database consistency
- Lower operational risk

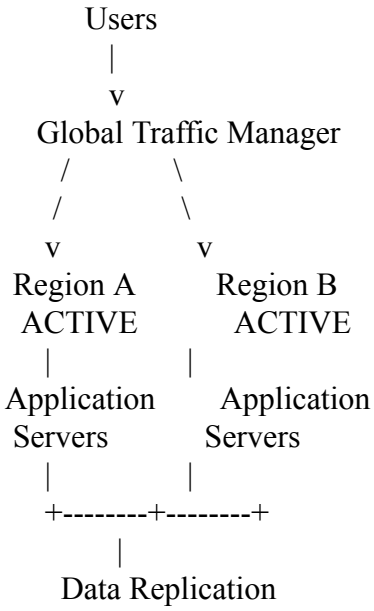
Disadvantages

- The secondary region may remain underutilized
- Failover can take some time
- Recovery may not be completely instantaneous

2. Active-Active Deployment

In an active-active model, both regions actively serve users at the same time.

Architecture



For example:

Users from Asia → Region A

Users from Europe → Region B

Both regions process requests simultaneously.

Advantages

- Very high availability
- Lower latency
- Faster failover
- Better resource utilization

- Supports global users

Disadvantages

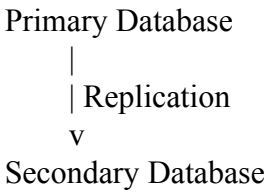
- More expensive
- More complex
- Database synchronization is difficult
- Possible data consistency conflicts

For a large SaaS provider requiring continuous service, an **active-active architecture** is generally preferred when the application and database can support it.

C. Data Replication Strategy

Data must be available in multiple regions. If Region A fails, Region B must have enough up-to-date data to continue operations.

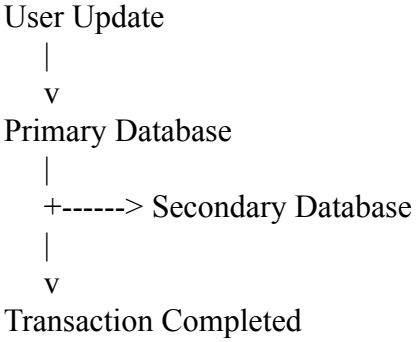
Database Replication



There are two main replication methods.

1. Synchronous Replication

In synchronous replication, data is written to both locations before the transaction is considered complete.



Benefits

- Very low risk of data loss
- Strong data consistency
- Lower RPO

Disadvantages

- Higher latency
- Performance may be slower, especially between distant regions

This method is useful for highly critical data.

2. Asynchronous Replication

In asynchronous replication, the primary database processes the transaction first, and the data is replicated later.

User Transaction
|
v
Primary Database
|
v
Transaction Completed
|
v
Replication to Secondary Database

Benefits

- Lower application latency
- Better performance
- Suitable for geographically distant regions

Disadvantages

- Some recent data may not be available if the primary region suddenly fails
- Small data loss is possible

D. Recommended Data Replication Configuration

The SaaS provider can use different replication methods depending on the importance of data.

For highly critical information:

Critical Data → Synchronous Replication

For less critical or high-volume data:

High-Volume Data → Asynchronous Replication

A typical design may include:

Region A Database
|
| Real-Time / Near Real-Time Replication
v
Region B Database
|
v
Backup Storage

The system should also monitor **replication lag**.

For example:

Primary Database Update
|
v
Secondary Database Update

Replication Lag = 2 Seconds

If the allowed RPO is 5 minutes, the replication lag must remain below the defined threshold.

E. Failover Mechanism

Failover is the process of automatically switching users and workloads from a failed region to a healthy region.

Failover Process

1. Region A is Operating Normally
|
v
2. Monitoring Detects a Problem
|
v
3. Health Checks Confirm Failure
|
v
4. Region A is Marked Unhealthy
|
v
5. Global Traffic Manager Redirects Traffic
|
v

6. Region B Becomes Primary
        |
        v
7. Application Continues Running

Health Checks

Health checks should continuously test:

- Application availability
- Server response
- Database connectivity
- API availability
- Network connectivity
- CPU and memory status

Example:

Health Check Request
      |
      v
Application Server
      |
  +----+----+
  | |
Healthy   Unhealthy
  | |
  v   v
Keep   Remove from
Traffic   Load Balancer

If a server or region repeatedly fails health checks, traffic should no longer be sent to it.

F. Global Traffic Failover

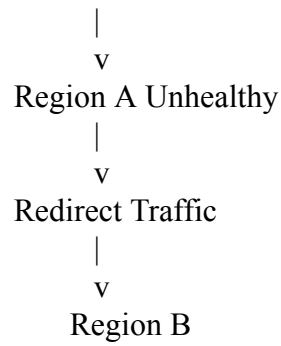
The Global DNS or Traffic Manager is responsible for directing users to a healthy region.

Normal Condition

Global Traffic Manager
      |
      v
   Region A

Region Failure

Global Traffic Manager

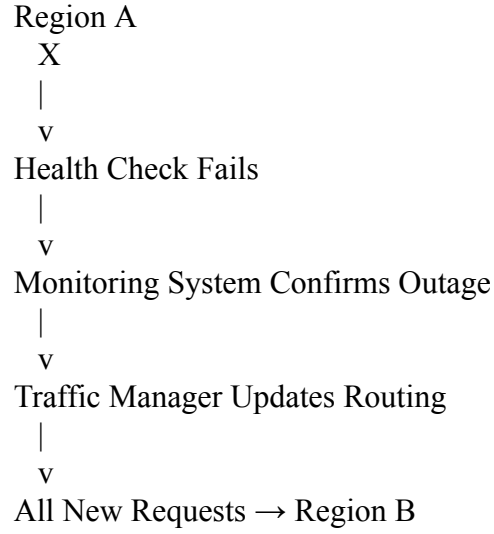


The traffic manager can make routing decisions based on:

- Region health
- Latency
- Geographic location
- Server load
- Network performance

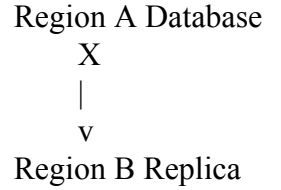
G. Automatic Failover Example

Suppose Region A experiences a complete outage.



The users should experience little or no interruption.

For stateful applications, the database replica in Region B may need to be promoted.



|
v
Promote to Primary Database

After Region A recovers, it can be synchronized again.

H. Failback Process

When the failed region becomes operational again, the system should not immediately move all traffic back without verification.

Failback Steps

1. Restore Region A infrastructure.
2. Check application health.
3. Synchronize data.
4. Verify database consistency.
5. Perform testing.
6. Gradually move traffic back if required.
7. Continue monitoring.

Region A Recovered
|
v
Synchronize Data
|
v
Health Verification
|
v
Gradual Traffic Restoration

Gradual failback reduces the risk of another outage.

I. Monitoring Strategy

The SaaS provider should implement centralized monitoring across all regions.

Monitoring should cover:

- Application servers
- Containers
- Databases
- Load balancers
- Storage
- Network connections

- Caches
- APIs
- Security systems
- Replication systems

Important Metrics

Application Metrics

- Response time
- Error rate
- Request rate
- Failed transactions
- API latency

Infrastructure Metrics

- CPU utilization
- Memory usage
- Disk usage
- Network throughput
- Server availability

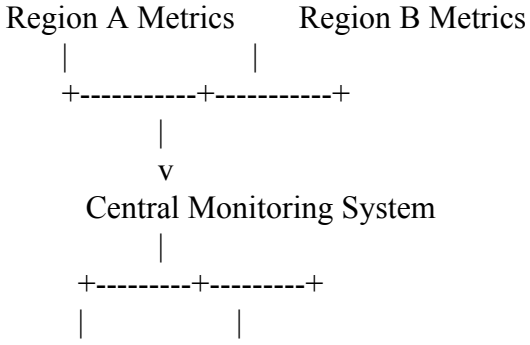
Database Metrics

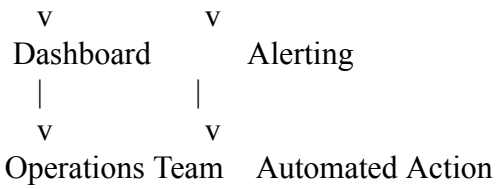
- Database connections
- Query response time
- CPU utilization
- Storage capacity
- Replication lag

User Experience Metrics

- Page load time
- API response time
- Failed login rate
- Transaction success rate

J. Centralized Monitoring Architecture





A centralized monitoring system gives administrators a complete view of the entire multi-region environment.

K. Alerting Strategies

Alerts should be generated before a complete failure occurs.

Example Alert Rules

IF CPU Usage > 80%
THEN Send Warning Alert

IF Error Rate > 5%
THEN Send Critical Alert

IF Response Time > 2 Seconds
THEN Notify Operations Team

IF Replication Lag > Allowed RPO
THEN Trigger Database Warning

IF Region Health Check Fails
THEN Start Automatic Failover

Alerts can be categorized as:

- Information
- Warning
- Critical

This prevents minor issues from being treated the same as major outages

L. Proactive Issue Detection

The monitoring system should identify problems before users experience a complete outage.

For example:

Response Time Increasing
|

v
Early Warning Detected
|
v
Investigate Application
|
v
Add More Resources
|
v
Prevent Service Failure

Common warning signs include:

- Gradually increasing response time
- Increasing error rates
- High CPU usage
- High memory usage
- Database overload
- Replication delays
- Network packet loss
- Low cache hit rate
- Unusual traffic pattern

M. Automated Recovery Actions

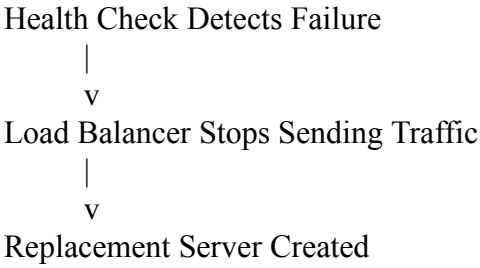
Monitoring can also trigger automated actions.

Example:

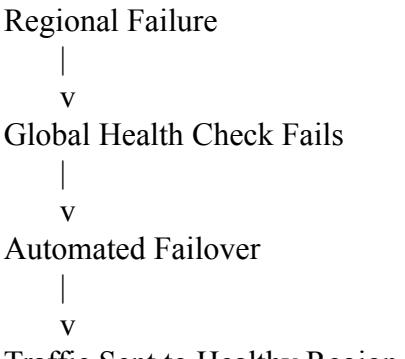
High CPU Usage
|
v
Monitoring Alert
|
v
Auto-Scaling Triggered
|
v
New Application Instances Created

Another example:

Application Server Failure
|
v



For a regional outage:

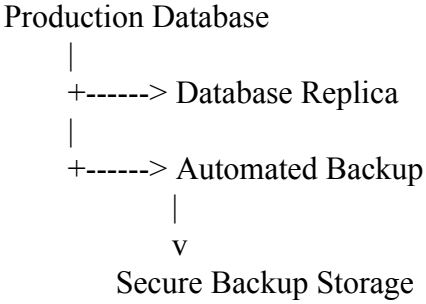


N. Backup and Disaster Recovery

Data replication alone is not sufficient. The provider should also maintain independent backups.

A recommended strategy includes:

- Automated database backups
- Point-in-time recovery
- Cross-region backup copies
- Versioning
- Immutable backups
- Regular disaster recovery testing



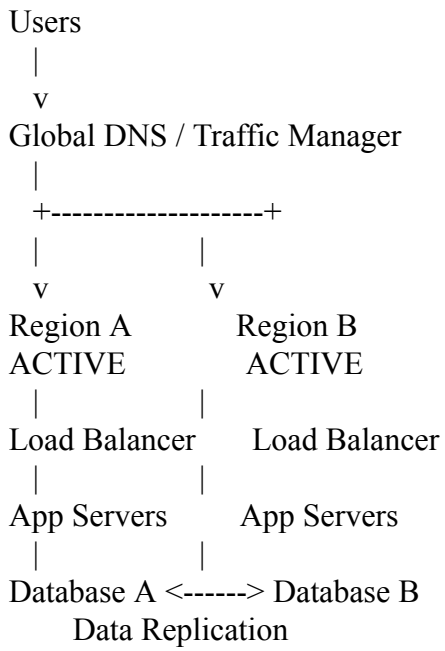
This protects against:

- Accidental deletion

- Data corruption
- Ransomware
- Replication errors
- Complete regional failurE

O. Recommended Final Multi-Region Strategy

A strong architecture can use:



The architecture should include:

1. Active-active deployment for high availability.
2. Global traffic management for intelligent user routing.
3. Regional load balancers for distributing traffic.
4. Replicated application servers across multiple availability zones.
5. Cross-region database replication.
6. Automated failover.
7. Continuous health checks.
8. Centralized monitoring.
9. Automated alerts.
10. Backup and disaster recovery systems.

Conclusion

To ensure high availability and fault tolerance, the SaaS provider should deploy the application across at least two geographically separate cloud regions. An active-active multi-region architecture provides the highest availability because both regions can serve users simultaneously.

Data should be replicated between regions using synchronous or asynchronous replication based on application requirements. Critical data may require synchronous replication for minimal data loss, while asynchronous replication can reduce latency for globally distributed workloads.

Automatic failover should use health checks, global traffic management, load balancers, and database replica promotion. If one region fails, the traffic manager should automatically redirect users to the healthy region.

For proactive issue detection, the organization should continuously monitor application performance, infrastructure health, database replication, error rates, response times, and network conditions. Automated alerts and recovery actions can identify and resolve issues before they cause major downtime.

Therefore, a combination of multi-region deployment, data replication, automatic failover, centralized monitoring, proactive alerting, backups, and disaster recovery provides a reliable, highly available, and fault-tolerant cloud architecture for a SaaS provider.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification Clarity	20% 15%	Decisions are well justified and technically strong Clear and well-organized response	Reasonable justification Generally clear	Weak justification Somewhat unclear	No useful justification Poorly presented